Inflation Money

2026-05-15

An Introduction

I've finally found time to check out Polymarket - a betting site with various subjects to bet on. I wanted to try and learn about time-series analysis, and an inflation bet seemed like an easy entry[1]

The Bet

I wanted to study and try out a bet with a few guiding principles:

1. Data is available and simple
2. The problem could be modeled using simple time-series methods

After some exploration, I've found the following simple finance question:

How high will inflation get in 2026?

With the following odds:

[1] in hindsight it's probably one of the toughest things to try and predict. The US government tries and fails at it constantly. Clearly, I'll manage to outsmart 'em.

We can see that inflation already crossed the 3% and 3.5% thresholds - they appear at the bottom as "resolved", i.e whoever bet on these won. It's important to understand the context of the bet - if at ANY month during the year inflation passes some threshold, it will be resolved in favor of whoever placed a bet on that threshold. In this problem's context, a given month's threshold is calculated using the *Consumer Price Index*. We need to get a little formal so we can start thinking about the problem:

Denote π_m the inflation ratio (abbreviated by IR from now on) at the end of month $m \in (1, ..., 12)$. Denote $t \in (4, 4.5, 5, 6, 8, 10) := T$ the critical thresholds suggested in the bet. Our event of interest is defined as $E_t := \{max_{m \in (1,...,12)} \pi_m \ > t\}$, for all thresholds t. We would like to model the probability $P(E_t)$ for all $t \in T$

The general process will be composed of 2 stages:

1. Collect, organize and analyze initial data
2. Construct a model designed to predict the probability $P(E_t)$

Step 1 - The Data

After understanding what we would like to predict, we can start building some basic models. First, we need to gather some data. I've downloaded data about past CPI information[2]. Each row is a single year, each column is a month. As always, we need to clean up the data before we can start using it effectively. The following code imports the data, and cleans it up so we can start processing it:

```r
data <- read_excel("hist_cpi-u.xlsx", skip = 3)
data <- data[-1, ] # remove first row
data <- data[, -1] # remove first col
data <- data %>%
  slice(1:(n()-3)) %>% # remove last 3 lines
  mutate(across(everything(),as.numeric)) # make sure all columns are numeric
```

here are the first couple of rows to get a feel for what we have:

```
## # A tibble: 6 x 13
##    Year  Jan.  Feb.  Mar.  Apr.   May  Jun.  Jul.  Aug.  Sep.  Oct.  Nov.  Dec.
##   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1  1913   9.8   9.8   9.8   9.8   9.7   9.8   9.9   9.9  10    10    10.1  10
## 2  1914  10     9.9   9.9   9.8   9.9   9.9  10    10.2  10.2  10.1  10.2  10.1
## 3  1915  10.1  10     9.9  10    10.1  10.1  10.1  10.1  10.1  10.2  10.3  10.3
## 4  1916  10.4  10.4  10.5  10.6  10.7  10.8  10.8  10.9  11.1  11.3  11.5  11.6
## 5  1917  11.7  12    12    12.6  12.8  13    12.8  13    13.3  13.5  13.5  13.7
## 6  1918  14    14.1  14    14.2  14.5  14.7  15.1  15.4  15.7  16    16.3  16.5
```

Step 2 - The Models

Brief Introduction

Now let's get our hands dirty with time-series analysis. In general, a time-series is just a sequence of data points indexed in chronological order. We can order our data in many different ways. One natural way to do so, is to go linearly in time. We can take $x_0 := PCI_{\text{January, 1913}}$, $x_1 := PCI_{\text{February, 1913}}$ and so on.

[2] https://www.bls.gov/cpi/tables/supplemental-files/

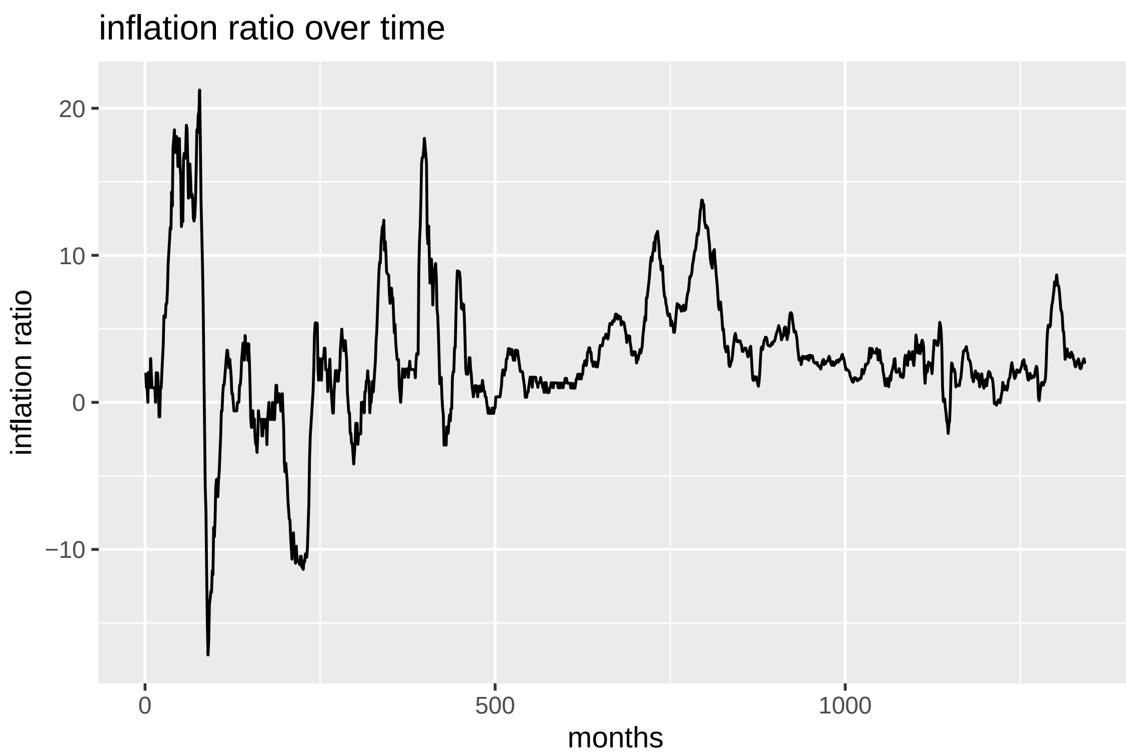
As we've seen earlier, what we're interested in is the inflation ratio - not the CPI of each month directly. From a short search in Google, we find the following relationship between IR and CPI at time t:

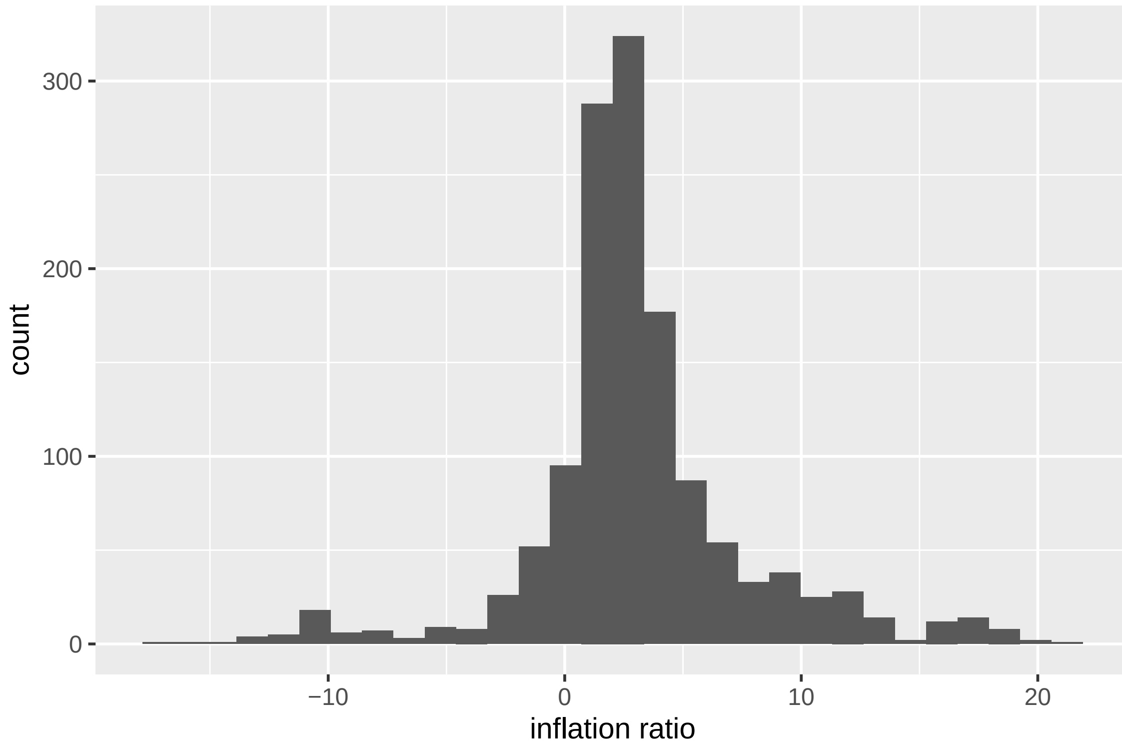
$$\text{inflation ratio at month t} := \pi_t = \frac{CPI_t - CPI_{t-12}}{CPI_{t-12}} \cdot 100 = (\frac{CPI_t}{CPI_{t-12}} - 1) \cdot 100$$

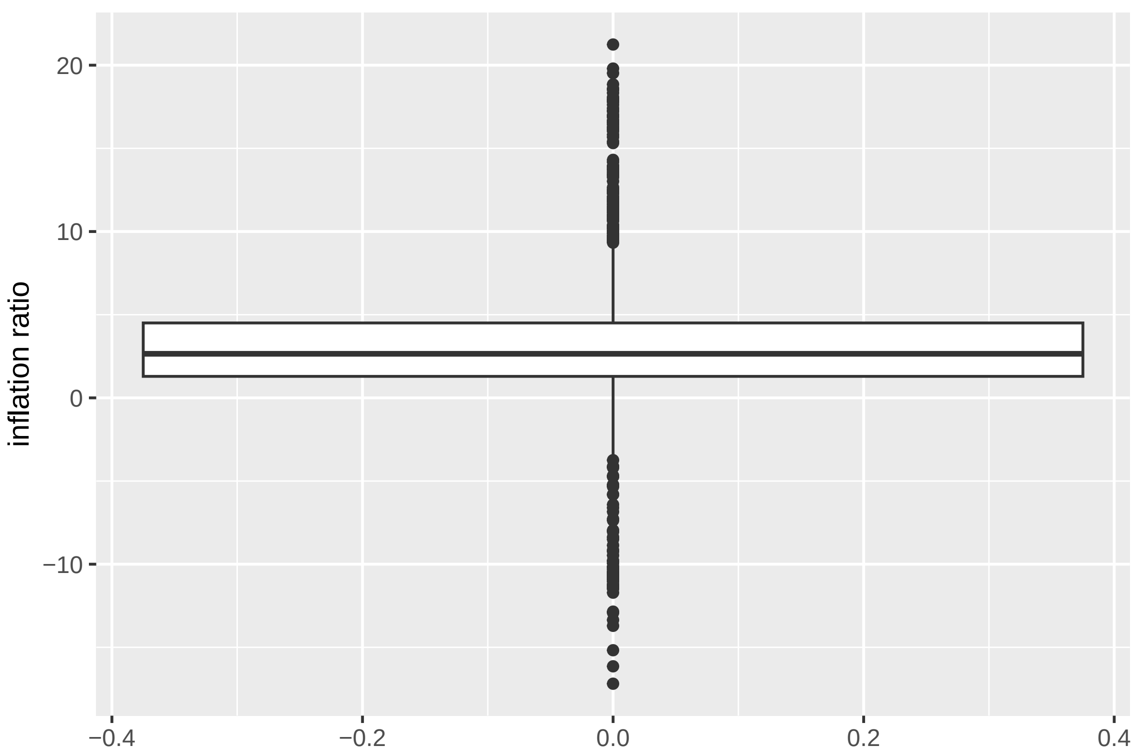
We should note that this is not a very comfortable variable, and a more suitable one could be $x_t := log(CPI_t) - log(CPI_{t-12}) = log(\frac{CPI_t}{CPI_{t-12}})$. First, notice the direct connection between π_t and x_t. if we denote $r_t := \frac{CPI_t}{CPI_{t-12}}$, we have $\pi_t = (r_t - 1) \cdot 100$ and $x_t = log(r_t)$. We can use Taylor series approximation $log(x+1) \approx x$ for small 'x's, and thus $log(r_t) = log(1 + (r_t - 1)) \approx r_t - 1$. Therefore: $\log(r_t) \approx \frac{\pi_t}{100} \rightarrow \pi_t \approx 100 \cdot log(r_t)$

We can therefore model x_t and easily find the connection back to π_t. log-scales are preferred for a couple of reasons - they are easy to manipulate algebraically, and represent proportional changes well. From now on, the variable we would like to predict is the log difference - defined simply as $x_t := log(CPI_t) - log(CPI_{t-12})$.

Let's start by composing the main time series vector $(x_0, x_1, ...)$ and see some basic visuals to get a feel for the trends in the data:







Model 1 - Auto-Regressive

We start off with a simple Auto Regressive model of order p, defined as:

$$x_t = \sum_{i=1}^{p} \phi_i x_{t-i} + \epsilon_t$$

The model has the following assumptions:

1. ϵ_t is white noise, i.e: $E[\epsilon_t] = 0$, $Var(\epsilon_t) = \sigma^2$ and $Cor(\epsilon_t, \epsilon_\tau) = 0$ for $t \neq \tau$
2. ϕ_i are constants, $i \in$ (1, … ,p)

There are a couple of things worth checking before making predictions:

1. is x_t a stationary process?
2. are the assumptions correct?
3. what is the optimal order p, considering our data?

is x_t stationary?

In non-technical terms, a stationary process is a process whose statistical properties don't change over time. This is an important property we wish our process will have, as it (roughly) gives us the ability to learn patterns from the past, and assume these patterns will continue in the future. There are a couple of definitions for stationary. We'll use stationary in the weak sense, i.e:

1. the mean value function $\mu_t = E[x_t]$ is constant, and doesn't depend on t
2. the auto-covariance function $\gamma(s,t) = cov(x_s, x_t)$ depends on s and t only through their difference $|s-t|$

We start with 2 tests which are standard to check whether a process is stationary or not, KPSS and ADF:

```
##
##  Augmented Dickey-Fuller Test
##
## data:  x_t
## Dickey-Fuller = -5.9926, Lag order = 11, p-value = 0.01
## alternative hypothesis: stationary
```

```
##
##  KPSS Test for Level Stationarity
##
## data:  x_t
## KPSS Level = 0.43603, Truncation lag parameter = 7, p-value = 0.06162
```

Both tests suggest stationary process - ADF's null hypothesis is that the process isn't stationary, while KPSS's null is that the process is stationary. Therefore, rejecting ADF and not rejecting KPSS suggests our time series is a stationary process.

To further check if our process is stationary, we can fit an Auto-Regressive model of order 1. If the AR(1) coefficient estimate is less than 1, we have more supporting evidence for a stationary process. We have:

```
## the AR(1) coefficient value is:  0.9866
```

This again implies stationary process. The value is indeed very close to 1, which suggests our case might be close to non-stationary. For now we'll proceed with assuming stationary process, but we remember we don't have the strongest indicators for it.

Now we wish to answer whether the assumptions of the AR model holds, and find the optimal order p. First we need to fit a model. We'll split the data to training, validation and test sets. Training will be used for model training. Validation will be used to pick optimal model order Test set will be estimate prediction error

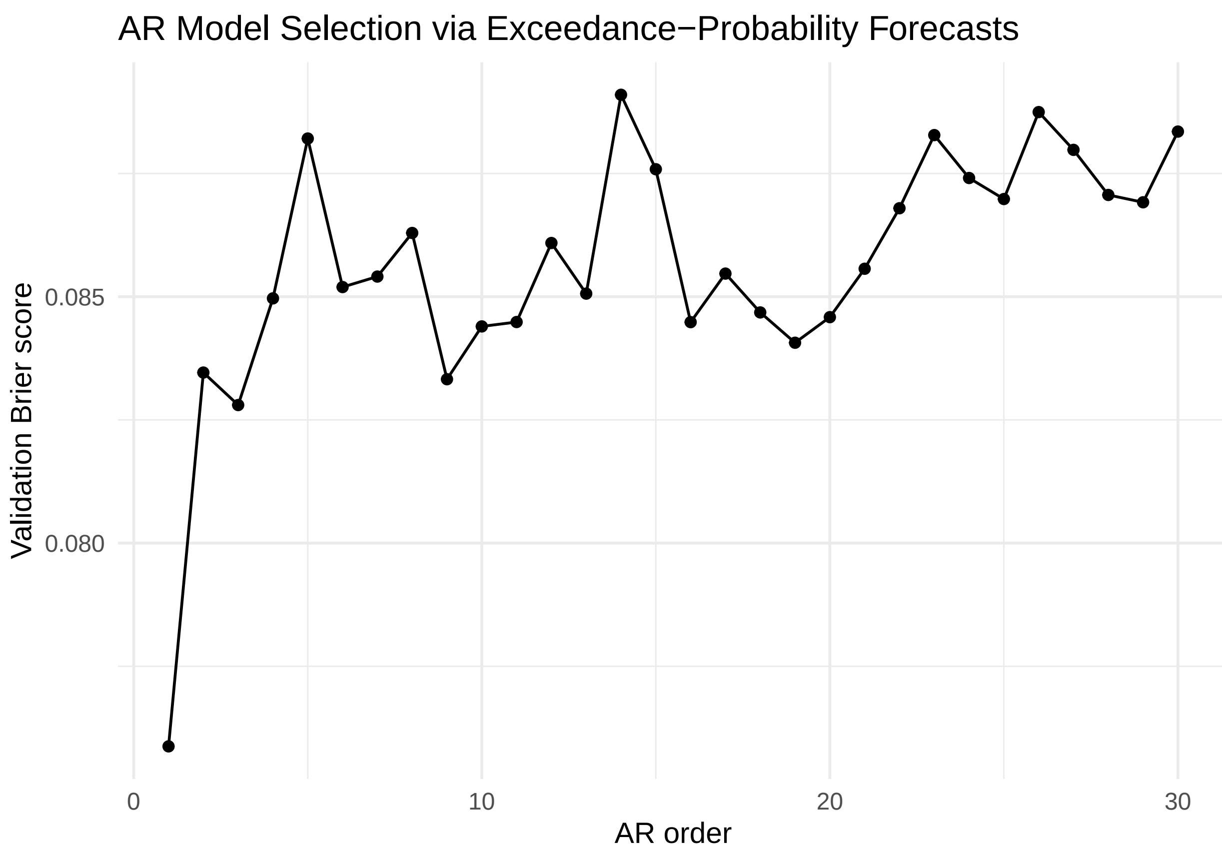
what is the optimal order p?

We start out by splitting the data into training, validation and test sets with 70/20/10 proportion. The training set will be used to train the model, validation set to tune the optimal order of our model to prevent over-fit, and the test set will be used to estimate prediction error. It's clear we can't just randomly split the data. If the test set is from the "past", and the learning set is from the "future", We'll have information leakage. So the first 70% points will be in the training set, the middle 20% will be the validation set and the last 10% the test set.

Now we need to understand how to find the optimal order p. A first basic question would be - optimal in what sense? In our problem, we're directly trying to estimate the probability that yearly inflation exceeds a threshold. Therefore, model validation should be performed on those probabilities. Here is the process we'll perform to choose an optimal order:

1. Fit AR models, from order 1 to 30
2. For each validation year:generate predictive distributions via simulation and estimate $P(E_t)$ for each threshold
3. Evaluate those probabilities with Brier score
4. Choose the order with the best average

Below we can see a plot which shows the Brier score for each model:


AR Model Selection via Exceedance−Probability Forecasts
Validation Brier score
0.085
0.080
0
10
20
30
AR order